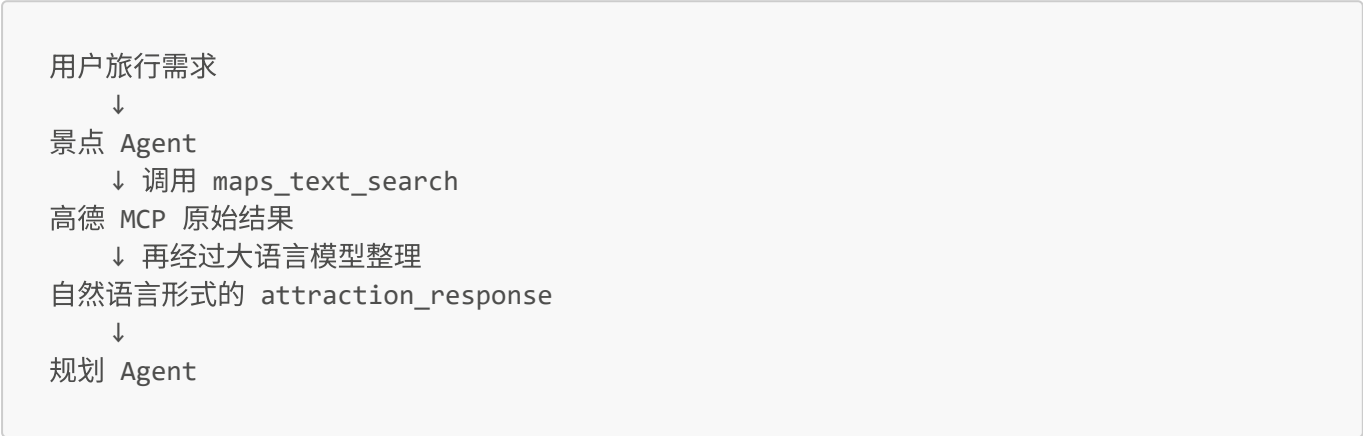


景点 Agent 数据闭环改造说明

文档日期：2026-09-20 适用项目：HelloAgents 智能旅行助手 改造范围：景点 Agent、高德 POI Service、规划 Agent、请求模型与异常处理 当前状态：景点核心数据闭环已经完成；通用行程校验、价格可信来源及多偏好召回仍待完善

1. 改造背景

改造前，景点 Agent 同时负责调用高德工具和整理景点信息。原始链路大致如下：

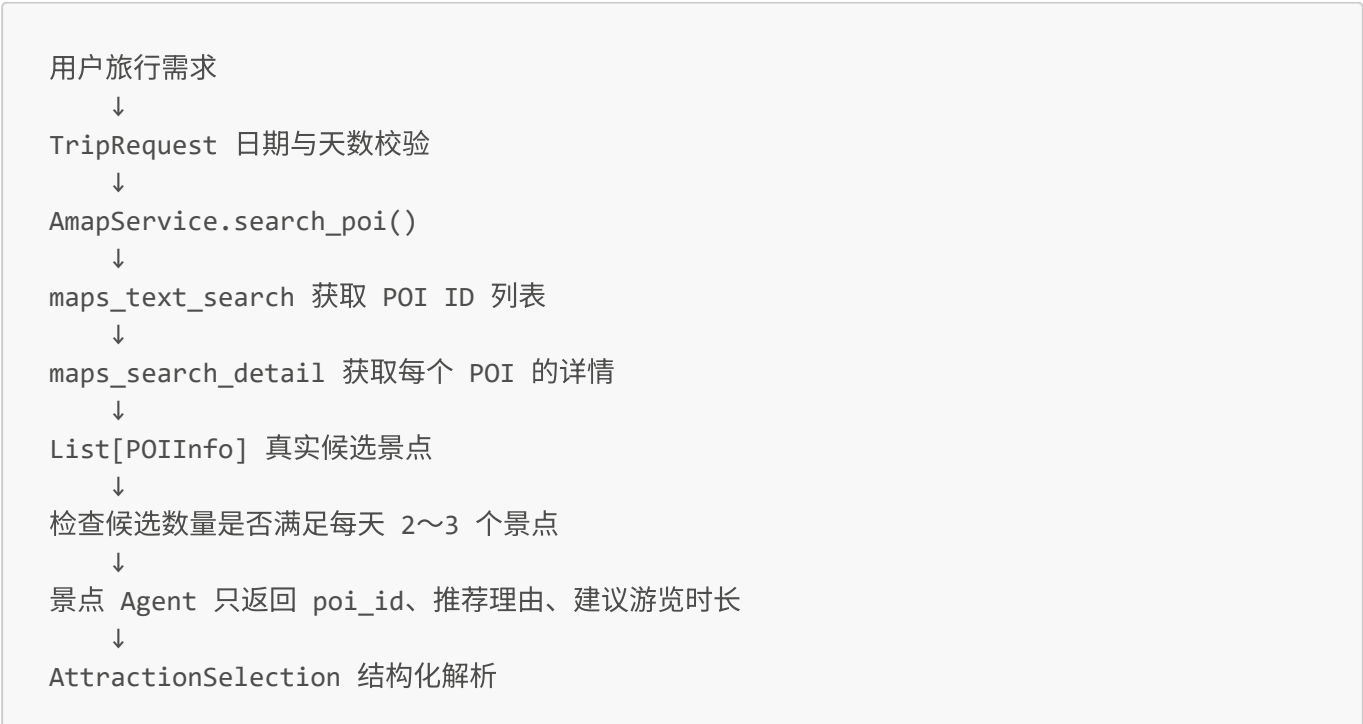


这种设计会把高德返回的客观事实与模型生成内容混在一起。即使高德结果正确，模型仍可能修改名称或地址、补造坐标、创建不存在的景点，或者修改 POI ID。

因此，本次改造不是单纯增加提示词约束，而是重新划分职责：

高德 Service 提供客观事实，景点 Agent 只负责从真实候选中选择，规划 Agent 只负责排程，Python 代码负责结构校验、白名单校验和可信字段回填。

2. 改造后的完整数据链路





职责边界如下：

数据或决策	负责人
POI ID、名称、地址、类型、坐标	高德 Service
推荐理由、建议游览时长	景点 Agent
景点安排在哪一天	规划 Agent
数据结构、数量、归属、重复检查	Python 代码

3. MCP 原始响应解析

3.1 问题

`MCPTool.run()` 通常返回带说明文字的字符串，而不是可以直接读取字段的字典：

```
工具 'maps_text_search' 执行结果：
{
  "pois": [...]
}
```

因此不能直接使用 `result["pois"]`。

3.2 通用解析器

项目新增：

```
backend/app/services/mcp_response_parser.py
```

`extract_json_value()` 会：

- 1. 直接接受已有的 `dict` 或 `list`；
- 2. 在字符串中寻找可能的 JSON 对象或数组；

3. 使用 `json.JSONDecoder.raw_decode()` 解析；
4. 返回 Python `dict` 或 `list`；
5. 找不到合法 JSON 时抛出 `MCPResponseParseError`。

它不依赖固定工具前缀，因此可以复用于不同 MCP 工具和 Agent 响应。

3.3 解析器测试

测试文件为 `backend/tests/test_mcp_response_parser.py`。当前七个测试覆盖：

- 带前缀文本中的 JSON 对象和数组；
- 已有 `dict` 和 `list`；
- 空字符串；
- 没有 JSON 的错误文本；
- 不支持的数据类型。

4. 高德 POI 事实数据转换

4.1 搜索与详情分离

`maps_text_search` 主要返回 POI ID、名称、地址和类型编码，通常没有项目需要的经纬度。例如：

```
{
  "id": "B000A8UIN8",
  "name": "故宫博物院",
  "address": "景山前街4号",
  "typecode": "110201|140100"
}
```

项目继续使用 POI ID 调用 `maps_search_detail`：

```
{
  "id": "B000A8UIN8",
  "name": "故宫博物院",
  "location": "116.397029,39.917839",
  "address": "景山前街4号",
  "city": "北京市",
  "type": "风景名胜;风景名胜;世界遗产",
  "rating": "4.9"
}
```

可信 POI 的构造过程为：

```
maps_text_search
  ↓ 提取 POI ID
maps_search_detail
```

↓ 解析完整详情

POIInfo

4.2 经纬度解析

高德坐标格式为 "116.397029,39.917839"，项目内部需要 `Location(longitude=..., latitude=...)`。

`parse_location()` 负责：

- 检查输入是否为字符串；
- 按逗号拆分经纬度；
- 转换为 `float`；
- 检查经度范围 `-180~180`；
- 检查纬度范围 `-90~90`；
- 返回 `Location`。

4.3 POI 详情转换

`parse_poi_detail()` 将高德详情字典转换为 `POIInfo`：

高德字段	项目字段	处理方式
<code>id</code>	<code>POIInfo.id</code>	必须存在
<code>name</code>	<code>POIInfo.name</code>	必须存在
<code>type</code>	<code>POIInfo.type</code>	转为字符串
<code>address</code>	<code>POIInfo.address</code>	转为字符串
<code>location</code>	<code>POIInfo.location</code>	使用 <code>parse_location()</code>
<code>tel</code>	<code>POIInfo.tel</code>	缺失时为 <code>None</code>

`POIInfo` 是进入 Agent 流程前的统一事实模型。

5. AmapService 的结构化查询

5.1 `get_poi_info()`

POI ID

↓ maps_search_detail

MCP 原始字符串

↓ extract_json_value()

dict

↓ parse_poi_detail()

POIInfo

`get_poi_info()` 面向项目内部结构化数据；旧的 `get_poi_detail()` 暂时继续服务现有详情 API。

5.2 `search_poi()`

改造后的 `search_poi()` 会：

1. 调用 `maps_text_search`;
2. 解析 MCP 响应并读取 `pois`;
3. 提取每条记录的 POI ID;
4. 调用 `get_poi_info()` 获取完整详情;
5. 返回 `List[POIInfo]`;
6. 单个详情失败时跳过该 POI;
7. 所有详情都失败时抛出异常。

当前最多处理前十条搜索结果：

```
for item in poi_items[:10]:
```

该限制用于控制详情调用次数和整体耗时。

6. 景点 Agent 的结构化职责

6.1 输出模型

```
class RankedAttraction(BaseModel):
    poi_id: str
    reason: str
    suggested_duration: int

class AttractionSelection(BaseModel):
    attractions: List[RankedAttraction]
```

- `poi_id` 引用高德真实候选;
- `reason` 表示主观推荐理由;
- `suggested_duration` 表示建议游览时间，范围为 30~480 分钟;
- `AttractionSelection` 为 Agent 输出提供固定顶层结构。

景点 Agent 不再输出名称、地址、类别和坐标，也不再注册高德 MCP 工具。

6.2 提示词约束

景点 Agent 已从“搜索专家”调整为“筛选专家”。提示词要求：

- 只能使用候选列表中存在的 POI ID;
- 不得创建、修改或猜测 POI ID;
- 不得修改名称、地址、类型和坐标;
- 只生成推荐理由和建议游览时长;
- 只返回 JSON，不返回 Markdown 或额外说明。

7. 日期、行程范围与数量约束

7.1 请求日期校验

`TripRequest` 使用 `model_validator(mode="after")` 检查：

- 日期是否为 `YYYY-MM-DD`；
- 结束日期是否早于开始日期；
- `travel_days` 是否等于首尾日期包含当天后的实际天数。

当前 `travel_days` 范围为 1~5 天。限制为五天，是因为目前最多只有十个候选，而行程要求每天至少两个且全程不重复。

7.2 候选数量

主流程计算：

```
minimum_required = request.travel_days * 2
maximum_required = min(
    len(attraction_candidates),
    request.travel_days * 3,
)
```

真实候选少于 `minimum_required` 时，流程立即抛出 `PlanValidationError`，不再继续调用 Agent。

7.3 Agent 选择数量

景点 Agent 的实际选择数量必须位于：

```
旅行天数 × 2
到
min(真实候选数量, 旅行天数 × 3)
```

提示词与 Python 校验使用同一组上下限。提示词负责引导，Python 代码负责保证。

8. 第一层白名单：验证景点 Agent

高德候选转换为索引：

```
candidate_by_id = {
    poi.id: poi
    for poi in attraction_candidates
}
```

景点 Agent 返回结果需要通过：

- 选择数量上下限检查；
- 重复 POI ID 检查；

- 候选集合之外的 POI ID 检查；
- Pydantic 字段和游览时长范围检查。

通过后建立：

```
selected_by_id = {
    item.poi_id: item
    for item in attraction_selection.attractions
}

selected_id_set = set(selected_by_id)
```

代码随后组合 `trusted_attractions`：

POIInfo 提供：POI ID、名称、地址、坐标、类别
景点 Agent 提供：推荐理由、建议游览时长

规划 Agent 接收序列化后的 `trusted_attractions_json`，不再接收未经验证的景点 Agent 原始文本。

9. 第二层白名单：验证规划 Agent

规划 Agent 生成 `TripPlan` 后，程序再次检查：

- 每天必须有 2~3 个景点；
- 每个景点必须包含 `poi_id`；
- `poi_id` 必须属于高德真实候选 `candidate_by_id`；
- `poi_id` 必须属于景点 Agent 实际选中的 `selected_id_set`；
- 同一个 POI 不能跨天重复安排。

两层白名单的职责不同：

```
candidate_by_id
    保证 POI 确实来自高德候选

selected_id_set
    保证 Planner 没有绕过景点 Agent 的筛选结果
```

所有业务违规统一抛出 `PlanValidationError`。

10. 最终可信字段回填

规划 Agent 通过第二层校验后，代码根据 `poi_id` 重新获取来源对象：

```
source_poi = candidate_by_id[attraction.poi_id]
```

```
selected = selected_by_id[attraction.poi_id]
```

最终字段来源为：

最终字段	回填来源
name	POIInfo.name
address	POIInfo.address
location	POIInfo.location
category	POIInfo.type
visit_duration	RankedAttraction.suggested_duration
description	RankedAttraction.reason

规划 Agent 只负责决定景点的日期和顺序，不能修改高德事实，也不能修改景点 Agent 已确认的推荐理由与游览时长。

ticket_price 当前没有可信外部来源，仍应视为模型估算值。

11. 失败语义与异常处理

11.1 移除虚假备用计划

原来的 `_create_fallback_plan()` 会在异常后生成“某城市景点1”等虚假景点，并使用固定坐标。该 fallback 已移除。

现在的原则是：

无法生成可信行程时明确失败，不用虚构数据伪装成功。

11.2 业务异常

项目新增 `backend/app/exceptions.py`：

异常	含义	HTTP 状态
ExternalServiceError	高德、MCP 或模型等外部服务不可用	503
AgentOutputError	Agent 输出无法解析或不符合结构	502
PlanValidationError	生成结果违反业务约束	502
TripPlanningError	旅行规划基础异常	500
未知异常	未分类的内部错误	500

前端已经兼容后端返回的结构化错误对象，不再直接展示内部异常字符串。

11.3 当前异常分类边界

最终规划 Agent 的非法 JSON 和 Pydantic 错误已经转换为 `AgentOutputError`，景点筛选模型调用失败会转换为 `ExternalServiceError`。

景点筛选响应的 `extract_json_value()` 和 `AttractionSelection.model_validate()` 目前尚未统一包装为 `AgentOutputError`。这类异常仍会进入通用 500，后续应补齐为统一的 502 语义。

12. 测试状态

当前测试文件包括：

```
backend/tests/test_mcp_response_parser.py
backend/tests/test_trip_failure_semantics.py
```

覆盖内容包括：

- MCP JSON 对象和数组解析；
- 已有字典和列表输入；
- 空响应、错误文本和非法类型；
- 最终规划 Agent 返回非 JSON；
- 最终规划 Agent 返回不符合 `TripPlan` 的结构。

本阶段人工验证还应关注：

- 合法日期与天数可以创建请求；
- 日期跨度与 `travel_days` 不一致时被拒绝；
- 候选不足时明确失败；
- Agent 选择数量不符合范围时明确失败；
- 重复、候选外或未选中的 POI 被拒绝；
- 最终事实来自高德，推荐字段来自景点 Agent；
- 服务或模型失败时不会返回虚假行程。

13. 已知限制与后续工作

13.1 只使用第一个旅行偏好

当前搜索关键词来自 `request.preferences[0]`。多个偏好尚未分别召回并按 POI ID 去重。

13.2 候选最多十个

`search_poi()` 当前最多查询十个详情。五天行程至少需要十个有效 POI，任何详情查询失败都可能导致候选不足。

13.3 通用行程校验尚未独立

当前已校验景点数量、归属和重复，但尚未建立独立的通用行程校验器。以下规则仍待完善：

- `TripPlan.days` 数量是否等于 `travel_days`；
- 每天日期是否连续且对应请求日期；
- `day_index` 是否从 0 连续递增；

- 顶层城市和日期是否被规划 Agent 修改；
- 每天是否具有完整餐饮和酒店；
- 预算分项与总计是否一致。

13.4 门票价格仍为估算

`POIInfo` 没有可信门票价格来源。接入真实价格数据前，`ticket_price` 不能描述为高德事实。

13.5 查询性能

POI 详情当前顺序查询。后续可考虑有上限的并发查询、详情缓存、按旅行天数决定候选数量，以及减少重复详情请求。

13.6 其他模块尚未闭环

本说明只代表景点链路。酒店、餐饮、路线和预算仍需分别建立“外部事实、Agent 选择、代码校验、事实回填”的闭环。

14. 本阶段完成情况

- ☒ MCP 原始字符串可以统一解析；
- ☒ 已确认搜索和详情接口的真实返回格式；
- ☒ 坐标可以转换为 `Location`；
- ☒ POI 详情可以转换为 `POIInfo`；
- ☒ `search_poi()` 返回真实 `List[POIInfo]`；
- ☒ 景点 Agent 不再直接调用高德工具；
- ☒ 景点 Agent 只返回 POI ID、理由和游览时长；
- ☒ 请求日期、天数及五天上限已经校验；
- ☒ 候选数量和 Agent 选择数量已经校验；
- ☒ 景点 Agent 输出使用 Pydantic 结构化解析；
- ☒ 重复 ID 和候选外 ID 已被拒绝；
- ☒ 规划 Agent 只接收可信景点 JSON；
- ☒ 规划结果同时经过候选与选中集合校验；
- ☒ 跨天重复和每日 2~3 个景点规则已经校验；
- ☒ 高德事实字段已经最终回填；
- ☒ 推荐理由和游览时长已经从景点 Agent 结果回填；
- ☒ 虚假 fallback 已移除；
- ☒ API 已区分外部服务、Agent 输出和计划校验错误；
- ☒ 前端已支持结构化错误消息；
- ☐ 景点选择响应的解析异常统一映射为 `AgentOutputError`；
- ☐ 建立独立的通用行程校验器；
- ☐ 支持多偏好召回和更长行程；
- ☐ 为门票价格增加可信来源或明确估算标识。

15. 涉及文件

文件	作用
<code>backend/app/services/mcp_response_parser.py</code>	从 MCP 或 Agent 原始字符串中提取 JSON

文件	作用
backend/tests/test_mcp_response_parser.py	验证通用 JSON 提取器
backend/tests/test_trip_failure_semantics.py	验证最终规划 Agent 的失败语义
backend/app/services/amap_service.py	查询、解析并构造真实 POI 数据
backend/app/models/schemas.py	定义请求、POI 和景点 Agent 结构模型
backend/app/exceptions.py	定义旅行规划业务异常
backend/app/agents/trip_planner_agent.py	候选筛选、两层白名单、规划和最终回填
backend/app/api/routes/trip.py	将业务异常映射为 HTTP 响应
frontend/src/services/api.ts	解析并展示结构化错误消息

16. 核心总结

本次改造的核心不是让模型“承诺不幻觉”，而是让模型没有修改关键事实的权限：

外部事实 → AmapService

主观筛选 → 景点 Agent

行程排程 → Planner Agent

结构、数量和归属校验 → Python 代码

最终可信覆盖 → Python 代码

景点 Agent 即使产生候选外 ID，也会被第一层白名单拒绝；规划 Agent 即使添加未选中的景点或修改事实字
段，也会被第二层白名单拒绝或被可信来源覆盖。失败时系统明确返回错误，不再用虚构景点伪装成功。